

#### IV. ÉMULATION D'UN SYSTEME LINUX GRACE A JSLINUX



Source : lyceum.fr

Pour manipuler sous Linux, on utilisera l'émulateur de système mis au point par Fabrice Bellard accessible à l'adresse :

<https://bellard.org/jslinux/vm.html?url=https://bellard.org/jslinux/buildroot-x86.cfg>

##### 1. LA MANUEL D'UTILISATION

La commande `man` (pour `manual`) permet d'obtenir la page du manuel d'utilisation d'une commande. S'avère utile pour connaître la syntaxe exacte de la commande et les options possible



Commande non implémentée sur le simulateur

##### 2. NOTION D'UTILISATEUR ET DE MACHINE

Qui suis-je (renvoie le nom de l'utilisateur) : `whoami`

Où suis-je (renvoie le dossier (*directory*) courant) ? `pwd`

Par défaut quand on ouvre un terminal on se trouve dans son dossier `$HOME`:

- `/home/username` pour l'utilisateur `username`
- `/root/` si vous êtes `root`, l'utilisateur administrateur du système.

**Exo 2.** <https://bellard.org/jslinux/vm.html?url=https://bellard.org/jslinux/buildroot-x86.cfg>



Ouvrez une console, vous devriez avoir quelque chose qui ressemble à cela :

```
Loading...

Welcome to JS/Linux (x86)

Use 'vflogin username' to connect to your account.
You can create a new account at https://vfsync.org/signup .
Use 'export_file filename' to export a file to your computer.
Imported files are written to the home directory.

[root@localhost ~]#
```

ci-dessus la console de l'utilisateur `root` qui utilise un ordinateur qui se nomme `localhost` (`root@localhost`).

Les commandes sont saisies après le signe `#`.

a. Essayer les commandes suivantes:

- `whoami`
- `hostname`

b. Créer votre login : « You can create a new account at <https://vfsync.org/signup> . »

c. La commande "`su`" (Switch User, appelée plus communément (à tort semble-t-il) "Super Utilisateur") permet d'ouvrir une session avec l'ID (IDentifiant) d'un autre utilisateur, ou de démarrer un nouveau shell de connexion.

```
su toto
```

d. Après avoir entré votre mot de passe, vous êtes connecté en tant qu'utilisateur "`toto`", vous êtes toujours dans le même répertoire de travail que précédemment et vous bénéficiez des réglages par défaut de l'utilisateur précédent (shell de connexion, variables d'environnement, permissions diverses...)

e. Pour revenir à l'utilisateur `root` : `exit`

### 3. SYSTÈME DE FICHIERS LINUX

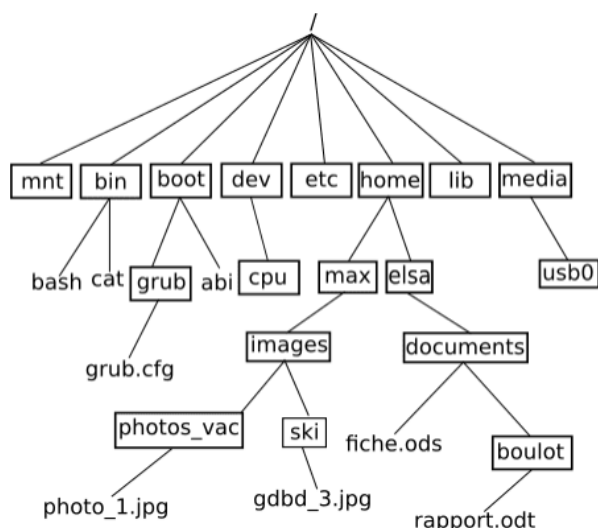
Sous Linux : pas d'unités logiques (C :,D :,...) comme sous Windows

Une seule arborescence dont la racine est désignée par le symbole /

Tous les volumes de données (Disque dur, lecteur USB,...) figurent dans cette arborescence

|        |                                                                          |
|--------|--------------------------------------------------------------------------|
| /      | racine de l'arborescence                                                 |
| /bin   | commandes binaires utilisateur essentielles (pour tous les utilisateurs) |
| /boot  | fichiers statiques du chargeur de lancement                              |
| /dev   | fichiers de périphériques                                                |
| /etc   | configuration système spécifique à la machine                            |
| /home  | répertoires personnels des utilisateurs                                  |
| /lib   | bibliothèques partagées essentielles et modules du noyau                 |
| /mnt   | point de montage pour les systèmes de fichiers montés temporairement     |
| /media | point de montage des clés USB, CD-ROM                                    |
| /proc  | système de fichiers virtuel d'information du noyau et des processus      |
| /root  | répertoire personnel de root                                             |
| /sbin  | binaires système (binaires auparavant mis dans /etc)                     |
| /tmp   | fichiers temporaires                                                     |

#### Ex 4 :



Dans le schéma ci-dessus on trouve des répertoires (noms entourés d'un rectangle, exemple : home) et des fichiers (uniquement des noms `grub.cfg`). À noter : les extensions des noms de fichiers, par exemple le `cfg` de `grub.cfg`, ne sont pas obligatoires dans les systèmes de type UNIX, par exemple, `bash` est bien un nom de fichier et il n'a pas d'extension.

👉 Pour indiquer la position d'un fichier (ou d'un répertoire) dans l'arborescence, chemin absolu ou chemin relatif ?

Le chemin absolu doit indiquer le chemin depuis la racine.

**Ex 5 :** le chemin absolu du fichier `fiche.ods` sera : `/home/elsa/documents/fiche.ods`

⚠ le chemin démarre bien de la racine / (attention les symboles de séparation sont aussi des /)

Le chemin relatif indique le chemin depuis le répertoire courant.

**Ex 6 :** Le chemin relatif permettant d'accéder au fichier `photo_1.jpg` depuis le répertoire `max` est :  
`images/photo_vac/photo_1.jpg`



L'absence du « / » au début du chemin indique que le départ est le répertoire courant.  
On peut aussi expliciter cet indication en utilisant « . / » au début du chemin.

**Ex 7 :** Pour indiquer le chemin relatif pour accéder au fichier `gdbd_3.jpg` depuis le répertoire `photos_vac` :



Il faut remonter d'un niveau dans l'arborescence pour se retrouver dans le répertoire `images` et ainsi pouvoir repartir vers la bonne branche. Pour ce faire il faut utiliser 2 points : `..`  
`../ski/gdbd_3.jpg`

**Rq1.** On peut remonter de plusieurs crans : `../..` depuis le répertoire `photos_vac` permet de remonter dans le répertoire `max`

**Exo 3.** En vous basant sur l'arborescence ci-dessus,

1. déterminer le chemin absolu permettant d'accéder au fichier :

- `cat` : `/bin/cat`
- `rapport.odt` : `/home/elsa/documents/boulot/rapport.odt`

2. Toujours en vous basant sur l'arborescence ci-dessus, déterminer le chemin relatif permettant d'accéder au fichier :

- `rapport.odt` depuis le répertoire `elsa` :  
`documents/boulot/rapport.odt` ou bien `./documents/boulot/rapport.odt`
- `fiche.ods` depuis le répertoire `boulot` :  
`../documents/fiche.ods` ou bien `../../documents/fiche.ods`



Attention à la casse : les systèmes de type UNIX sont sensibles à la casse (il différencie les caractères majuscules et les caractères minuscules) : le répertoire `Documents` et le répertoire `documents` sont 2 répertoires différents.

#### a) Quelques commandes usuelles

La commande `cd` (pour *change directory*) permet de changer le répertoire courant. Il suffit d'indiquer le chemin (relatif ou absolu) qui permet d'atteindre le nouveau répertoire :

La commande `ls` permet de **lister** les fichiers et répertoires du dossier courant

**Ex 8 :**

- `ls /chemin/vers/dossier`: liste les fichiers et répertoires du dossier donné en argument/
- `cd /root`

**Ex 9 :** En utilisant l'arborescence ci-dessus :

- si le répertoire courant est le répertoire `elsa` et que vous voulez vous rendre dans le répertoire `documents`, il faudra saisir la commande : `cd documents` (relatif) ou `cd /home/elsa/documents` (absolu)
- si le répertoire courant est le répertoire `photos_vac` et que vous voulez vous rendre dans le répertoire `ski`, il faudra saisir la commande : `cd ../ski` (relatif) ou `cd /home/max/images/ski` (absolu)

- si le répertoire courant est le répertoire `boulot` et que vous voulez vous rendre dans le répertoire `documents`, il faudra saisir la commande : `cd ..` (relatif) ou `cd /home/elsa/documents` (absolu)

**Rq2.**

3. visualiser le contenu d'un fichier : `cat nom_du_fichier`
4. Créer un dossier : `mkdir chemin vers le/dossier`

**Exo 4.** Après avoir ouvert une console, utilisez la commande `ls` depuis votre répertoire personnel.

La commande `mkdir` permet de créer un répertoire dans le répertoire courant. La commande est de la forme `mkdir nom_du_répertoire`

**Rq3.** Remarque : il est préférable de ne pas utiliser de caractères accentués dans les noms de répertoire (ou de fichier). Il en est de même pour les espaces (à remplacer par des caractères tirets bas `_`)

**Exo 5.** Après avoir ouvert une console, utilisez la commande `mkdir` afin de créer un répertoire `test_nsi` dans votre répertoire personnel.

```
[elsa@localhost ~]$ ls
[elsa@localhost ~]$ mkdir test_nsi
[elsa@localhost ~]$ ls
test_nsi
[elsa@localhost ~]$
```

La commande `rm` permet de supprimer un fichier ou un répertoire. La commande est de la forme `rm nom_du_répertoire_ou_nom_du_fichier`, pour **supprimer un répertoire non vide**, utiliser `rm -rf ...`

**Rq4.** La plupart des commandes UNIX peuvent être utilisées avec une ou des options. Par exemple, pour supprimer un répertoire non vide, il est nécessaire d'utiliser la commande `rm` avec l'option `-r` : `rm -r nom_du_répertoire`

La commande `touch` permet de créer un fichier vide. La commande est de la forme `touch nom_du_fichier_à_créer`

La commande `cp` permet de copier un fichier. La commande est de la forme `cp /répertoire_source/nom_fichier_à_copier /répertoire_destination/nom_fichier`

**Exo 6.** Placez-vous dans le répertoire `test_nsi` créé précédemment.

Créez un fichier `test.txt`.

Créez un répertoire `doc`.

```
[elsa@localhost ~]$ pwd
/home/elsa
[elsa@localhost ~]$ cd test_nsi
[elsa@localhost test_nsi]$ pwd
/home/elsa/test_nsi
[elsa@localhost test_nsi]$ touch test.txt
[elsa@localhost test_nsi]$ ls
test.txt
[elsa@localhost test_nsi]$ mkdir doc
[elsa@localhost test_nsi]$ ls
doc      test.txt
[elsa@localhost test_nsi]$
```

Copiez le fichier `test.txt` dans le répertoire `doc`.

```
[elsa@localhost test_nsi]$ cp test.txt doc
[elsa@localhost test_nsi]$ ls
doc      test.txt
[elsa@localhost test_nsi]$ cd doc
[elsa@localhost doc]$ pwd
/home/elsa/test_nsi/doc
[elsa@localhost doc]$ ls
test.txt
```

Effacez le répertoire doc (et son contenu).

```
[elsa@localhost doc]$ cd ..
[elsa@localhost test_nsi]$ pwd
/home/elsa/test_nsi
[elsa@localhost test_nsi]$ ls
doc      test.txt
[elsa@localhost test_nsi]$ rm -rf doc
[elsa@localhost test_nsi]$ ls
test.txt
[elsa@localhost test_nsi]$
```

## b) Droits des utilisateurs et groupes

L'option `-l` (pour *long*) passée à la commande `ls` permet de connaître les **droits** sur les fichiers



La commande `ls -l` précise dans sa sortie les attributs suivants de chaque fichier :

permission type utilisateur propriétaire groupe propriétaire taille date nom

```
ls -l Documents/git/2heoinfo/flask-csv/

total 28
-rw-r--r-- 1 ben docker 2690  4 juil. 10:39 brouillon.ipynb
-rw-r--r-- 1 ben docker  120  4 juil. 12:30 data.csv
-rw-r--r-- 1 ben docker  881  4 juil. 13:53 flask-csv.py
-rw-r--r-- 1 ben docker 1064  4 juil. 13:42 LICENSE
drwxr-xr-x 2 ben docker 4096  2 juil. 22:26 pages
-rw-r--r-- 1 ben docker  501  4 juil. 12:39 README.md
-rw-r--r-- 1 ben docker   70  4 juil. 10:07 run-server.sh
```

Les droits sont sous la forme `drwxrwxrwx`.

- `d` indique qu'il s'agit d'un dossier
- `r`: droit de lecture
- `w`: droit d'écriture
- `x`: droit d'exécution



Les droits sont attribués dans l'ordre à l'utilisateur (*user*), au groupe (*group*), aux autres (*others*).

**Ex 10 :** `-rw-r--r--`: fichier avec droit de lecture et d'écriture pour l'utilisateur propriétaire, mais uniquement des droits de lecture pour les membres du groupe propriétaire et les autres utilisateurs.

**Ex 11 :** `drwxr-xr-x`: dossier avec droit de lecture, écriture et exécution pour l'utilisateur propriétaire, mais uniquement des droits de lecture et d'exécution pour les membres du groupe propriétaire et les autres utilisateurs.

La commande `chmod` (*change mode*) permet de gérer les droits des fichiers et répertoires



Un utilisateur a le droit de faire un `chmod` sur un fichier :

- s'il est `root` ;
- ou s'il est le propriétaire du fichier en question.



Les options passées à la commande `chmod` sont indiquées de la manière suivante :

`chmod options modes fichiers`

Pour un fichier : `chmod [u g o a] [+ - =] [r w x] nom_du_fichier`



Pour le contenu d'un répertoire (de façon *réursive*) :

`chmod -R [u g o a] [+ - =] [r w x] nom_du_répertoire`

Les lettres `u g o a` désignent les types de propriétaires concernés :

- `u` propriétaire (*user*)
- `g` groupe (*group*)
- `o` les autres (*others*)
- `a` tous (*all*)



Les lettres `+` `-` `=` désignent les changements d'état: `+` et `-` pour ajouter ou retirer un type de droit aux droits courants, et l'opérateur `=` pour les écraser.

Les lettres `r w x` désignent les permissions : *read*, *write*, *execute*.

**Ex 12 :** Dans le cas précédent si l'on souhaite que tous les utilisateurs du groupe `docker` puissent lire et écrire dans le fichier `brouillon.ipynb`, on lance la commande `chmod g+w` sur le fichier:

```
chmod g+w Documents/git/2heoinfo/flask-csv/brouillon.ipynb
```



On peut alors vérifier le changement de droits avec `ls -l`

```
ben@coolerarch:~$ ls -l Documents/git/2heoinfo/flask-csv/
total 28
-rw-rw-r-- 1 ben docker 2690  4 juil. 10:39 brouillon.ipynb
...
```

La commande `chown` (*change owner*) permet de gérer les propriétaires des fichiers et répertoires



L'utilisateur racine(`root`) peut changer le propriétaire d'un fichier ou d'un dossier

La syntaxe générale de la commande est :

`chown [-hLPR] [utilisateur][:groupe] cible1 [cible2 ..]`

- Rendre l'utilisateur `john` propriétaire du fichier `toto.txt`: `chown john /home/jane/toto.txt`
- Pour modifier le propriétaire de tout le contenu d'un dossier : `chown -Rf utilisateur1 /home/dossier1`
- Pour modifier l'utilisateur et le groupe : `chown ben:prof /home/jane/toto.txt`

### Exo 7.

- a. Dans la capture ci-dessous, analyser les droits utilisateurs du fichier `photo.jpg`

```
david@david-XPS-13-9343 ~/nsi $ ls -l
total 4
-rw-r--r-- 1 david david    0 avril 13 19:58 fic.txt
drwxr-xr-x 2 david david 4096 avril 13 20:05 info
-rw-r--r-- 1 david david    0 avril 12 09:19 photo.jpg
david@david-XPS-13-9343 ~/nsi $
```



`fic.txt` est un fichier en lecture/écriture pour l'utilisateur propriétaire `david`, et lecture seulement pour le groupe propriétaire `david`, et pour les autres utilisateurs, de taille 0 octets.

- info est un dossier (ou répertoire) en lecture (on peut en lister les fichiers ou sous répertoires) écriture (on peut créer renommer ou supprimer des fichiers de ce répertoire) et exécutable signifie pour un répertoire que l'on peut en faire le répertoire courant (avec `cd info`) et afficher les infos détaillées de ce répertoire (avec `ls -l`)
- photo.jpg est un fichier en lecture/écriture pour l'utilisateur propriétaire david, et lecture seulement pour le groupe propriétaire david, et pour les autres utilisateurs, de taille 0 octets.

b. Analysez attentivement l'enchaînement de commandes suivantes :

```
david@PC-Bureau ~ $ cd nsi
david@PC-Bureau ~/nsi $ ls
david@PC-Bureau ~/nsi $ touch toto.txt
david@PC-Bureau ~/nsi $ ls -l
total 0
-rw-rw-r-- 1 david david 0 avril 21 10:20 toto.txt
david@PC-Bureau ~/nsi $ chmod o+x toto.txt
david@PC-Bureau ~/nsi $ ls -l
total 0
-rw-rw-r-x 1 david david 0 avril 21 10:20 toto.txt
david@PC-Bureau ~/nsi $ chmod g-w toto.txt
david@PC-Bureau ~/nsi $ ls -l
total 0
-rw-r--r-x 1 david david 0 avril 21 10:20 toto.txt
david@PC-Bureau ~/nsi $
```

- `chmod o+x toto.txt` : modifie les droits des autres utilisateurs en rendant le fichier `toto.txt` exécutable.
- `chmod g-w toto.txt` : modifie les droits des utilisateurs du groupe en leur supprimant le droit en écriture sur le fichier `toto.txt`.

c. Créez un répertoire `test_nsi2` dans votre répertoire personnel. Placez-vous dans le répertoire `test_nsi2`. Créez un fichier `titi.txt`, vérifiez les permissions associées à ce fichier.

```
[elsa@localhost test_nsi]$ cd
[elsa@localhost ~]$ pwd
/home/elsa
[elsa@localhost ~]$ mkdir test_nsi2
[elsa@localhost ~]$ cd test_nsi2
[elsa@localhost test_nsi2]$ pwd
/home/elsa/test_nsi2
[elsa@localhost test_nsi2]$ touch titi.txt
[elsa@localhost test_nsi2]$ ls -l
total 0
-rw-r--r-- 1 elsa input 0 May 13 2020 titi.txt
```

Modifiez les permissions associées au fichier `titi.txt` afin que les autres utilisateurs aient la permission écriture

```
[elsa@localhost test_nsi2]$ chmod o+w titi.txt
[elsa@localhost test_nsi2]$ ls -l
total 0
-rw-r--rw- 1 elsa input 0 May 13 2020 titi.txt
```

## V. Exos



Pour une synthèse des commandes, voir <https://juliend.github.io/linux-cheatsheet/>

**Exo 8.** On suppose que le répertoire personnel de l'utilisateur courant est vide.

- Décrire l'effet de chacune des commandes successives suivantes
- Ouvrir un terminal et vérifier que tout se passe comme prévu.

- `cd ~` : on va à la racine du répertoire personnel de l'utilisateur
- `mkdir NSI` : on crée le répertoire `NSI` dans le répertoire courant

- mkdir NSI/TP\_SHELL : dans le répertoire NSI, on crée le répertoire TP\_SHELL
- cd NSI/TP\_SHELL : on va dans le répertoire nouvellement créé

```
[elsa@localhost test_nsi2]$ cd ~
[elsa@localhost ~]$ pwd
/home/elsa
[elsa@localhost ~]$ mkdir NSI
[elsa@localhost ~]$ mkdir NSI/TP_SHELL
[elsa@localhost ~]$ ls
NSI      test_nsi  test_nsi2
[elsa@localhost ~]$ cd NSI/TP_SHELL
[elsa@localhost TP_SHELL]$ pwd
/home/elsa/NSI/TP_SHELL
```

- cd .. : on retourne dans NSI
- ls : on liste les fichiers et les répertoires (il ne devrait y avoir que le répertoire TP\_SHELL)

```
[elsa@localhost TP_SHELL]$ cd ..
[elsa@localhost NSI]$ pwd
/home/elsa/NSI
[elsa@localhost NSI]$ ls -l
total 4
drwxr-xr-x  2 elsa  input      37 May 13  2020 TP_SHELL
```

- chmod u+rwx,g-rwx,o-rwx TP\_SHELL :  
on rend le répertoire TP\_SHELL :  
\* lisible, scriptible (équivalent français de « écrivable » qui est un mot qui n'existe pas...) et exécutable à l'utilisateur,  
\* et totalement invisible au groupe et aux autres utilisateurs.

```
[elsa@localhost NSI]$ chmod u+rwx,g-rwx,o-rwx TP_SHELL
[elsa@localhost NSI]$ ls -l
total 4
drwx-----  2 elsa  input      37 May 13  2020 TP_SHELL
```

### Exo 9. Motifs glob : source [http://wiki.linux-france.org/wiki/Les\\_commandes\\_fondamentales\\_de\\_Linux](http://wiki.linux-france.org/wiki/Les_commandes_fondamentales_de_Linux)

Nous pouvons demander à ls de lister uniquement les fichiers dont les noms contiennent des caractères donnés. Gagnons le répertoire "/bin": cd /bin puis demandons par exemple la liste des seuls noms de fichiers dont les noms commencent par la lettre "g": ls g\*

L'argument "g\*" que nous venons de fournir est un motif, auquel on peut penser comme à une sorte de filet dont les mailles sont définies par nous (utilisateur), en respectant un certain nombre de conventions, et qui est ensuite appliqué (jeté) par le shell afin de sélectionner des noms de fichiers qui y correspondent.

Les principales conventions valides dans ce contexte, donc lorsque l'on définit un motif de sélection de noms de fichiers destiné au shell, sont :

l'étoile (ou plutôt l'astérisque) est un caractère générique sélectionnant **n'importe quelle séquence de caractères** (potentiellement vide)

le point d'interrogation sélectionnant **n'importe quel caractère**.

[c1-cn] sélectionne tous les caractères contigus de la plage de c1 à cn.

- Pour chacun des motifs glob ci-dessous donner une suite de caractères (de longueur >=1) reconnue par le motif :
  - \*.jpg :  
toute chaîne suivie de « .jpg » (en général pour désigner tout fichier dont l'extension est jpg)
  - \*.??? :  
toute chaîne suivie de « . », puis de trois caractères (en général pour désigner tout fichier avec extension de longueur 3)
  - ????? :  
toute chaîne de 5 caractères
  - ?\*? :  
toute chaîne d'au moins 2 caractères, par ex : ab, 1abcd\_e, etc.

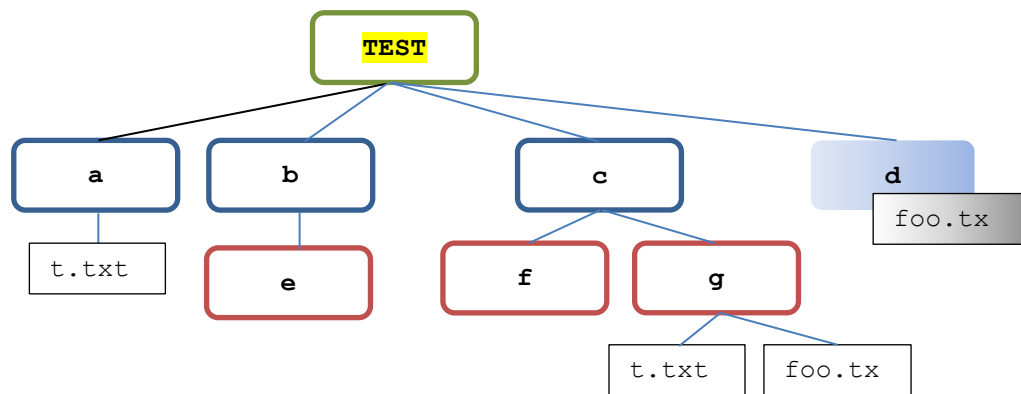
- `???0.???*` :  
toute chaîne de 3 caractères suivie d'un zéro et d'un point puis d'une chaîne d'au moins 3 caractères. par ex : `abc0.xyz` ou `def0.tuvw`
  - `[0-9]*[a-z]` :  
toute chaîne commençant par un chiffre et se terminant par une lettre minuscule : par ex, `2abCDe`, ou `0FGHgrttttijk`, ou `3k`
  - `[A-Z]??*.j*g`: toute chaîne commençant par une lettre MAJUSCULE, suivie d'une chaîne d'au moins 2 caractères, puis d'un point et d'une chaîne commençant par j et finissant par g : par ex, `AbCDzzzo.jpg`, ou `Abo.jpeg`
  -
2. Tester les réponses dans un terminal. Pour cela, se placer dans le répertoire `TP_SHELL` créé lors de l'exercice précédent, créer les fichiers vides au moyen de la commande « `touch nom du fichier` » et tester l'expression au moyen de « `ls motif` ». Le fichier créé avec touch doit être listé (au moins lui).

Par ex. :

```
[elsa@localhost NSI_TEST]$ ls
[elsa@localhost NSI_TEST]$ mkdir TP_SHELL
[elsa@localhost NSI_TEST]$ cd TP_SHELL
[elsa@localhost TP_SHELL]$ touch unfichier.txt
[elsa@localhost TP_SHELL]$ ls *txt
unfichier.txt
```

**Exo 10.** On suppose que l'on se trouve dans un répertoire `TEST`, vide, et que l'on exécute les commandes successives suivantes :

- `mkdir a b c d`
  - `touch a/t.txt d/foo.tx`
  - `cd c`
  - `mkdir ../b/e f g`
  - `cd ..`
  - `cp */*.txt c/g`
  - `rm -rf d`
- arborescence, avant la commande `rm -rf d`



Arborescence finale :

